Numerical analysis

Numerical analysis is concerned with obtaining approximate solutions to mathematical problems that cannot be solved exactly using analytical methods. It plays a vital role in engineering, physics, optimization, finance, weather prediction, machine learning, artificial intelligence, and many other scientific disciplines. SageMath provides a comprehensive environment for numerical computation by integrating several powerful libraries, including GSL (GNU Scientific Library), MPFR, MPFI, Arb, mpmath, SciPy, and NumPy. These libraries enable SageMath to perform high-precision arithmetic, numerical integration, root-finding, optimization, and a wide range of other numerical computations efficiently.

Let us now look at a few SageMath one-liners related to numerical analysis. The function *find_root(f, a, b)* numerically computes a root of the function *f* within the interval *[a,b]*. The function *numerical_integral(f, a, b)* numerically evaluates the definite integral of *f* over the interval *[a,b]*. Unlike symbolic integration, which attempts to derive an exact mathematical expression, numerical integration computes an approximate value using efficient numerical algorithms. The function returns a pair of values: the first is the computed value of the integral, while the second is an estimate of the numerical error. Finally, the function *find_local_maximum(f, a, b)* numerically determines a local maximum of the function *f* over the specified interval. It returns a pair consisting of the maximum value of the function and the point at which this maximum occurs. Figure 2 shows the output produced by these SageMath one-liners.

Let us now try to understand the output shown in Figure 2. The output of the function *find_root( )* indicates that the equation $x^3-x-2=0$ has a solution in the interval *[1, 2]* that is approximately equal to 1.52138. Substituting this value into the equation yields a result that is very close to zero, confirming that it is indeed a root of the equation. The output of the function *numerical_integral( )* shows that the numerical value of the integral of *sin(x)* over the interval *[0, π]* is approximately 1.9999999999999998, which is extremely close to the exact value of 2. The second value in the output is an estimate of the numerical error. Since this value is extremely small ($2.2204460492503128\times10^{-14}$), the computed result is highly accurate. Finally, the output of the function *find_local_maximum( )* indicates that the maximum value of *tan(x)* over the interval *[0, π/3]* is approximately 1.7320506864, attained at 1.0471975209 radians, which is approximately π/3. Since the tangent function is monotonically increasing over this interval, the maximum naturally occurs at the right endpoint. Consequently, the computed maximum is very close to the exact value, tan(π/3) = √3 ≈1.7320508076.

```
sage: find_root(x^3 - x - 2, 1, 2)
1.5213797068045676
sage: numerical_integral(sin(x),0,pi)
(1.9999999999999998, 2.2204460492503128e-14)
sage: find_local_maximum(tan(x),0,pi/3)
(1.7320506864202516, 1.0471975209094397)
```

Figure 2: A few SageMath one-liners for numerical analysis

Statistics

Statistics is the science of collecting, analysing, interpreting, and presenting data. It plays a central role in scientific research, data science, artificial intelligence, finance, healthcare, and numerous other fields where informed decisions are driven by data. SageMath provides a powerful environment for statistical computing by integrating widely used libraries such as NumPy, SciPy, and R, enabling users to perform a broad range of statistical analyses through a unified interface. Let us now look at a few SageMath one-liners for statistical computing, shown in Figure 3. Notice that these one-liners are standard Python statements, once again demonstrating that SageMath seamlessly integrates with the vast Python ecosystem while providing a rich mathematical environment for scientific computing.

```
sage: import numpy as np
sage: np.mean([2,4,6,8,10])
6.0
sage: np.var([2,4,6,8,10])
8.0
sage: np.std([2,4,6,8,10])
2.8284271247461903
```

Figure 3: A few SageMath one-liners for statistics

Let us now understand the output produced by these statistical functions. The function *np.mean( )* computes the arithmetic mean of the given data. For the list [2,4,6,8,10], the mean is 6.0, which is obtained by dividing the sum of the numbers (30) by the total number of observations (5). The function *np.var( )* computes the population variance, which measures how far the observations are spread around their mean. For the given data, the variance is 8.0. Finally, the function *np.std( )* computes the population standard deviation, which is the square root of the variance. The output 2.8284271247461903 is approximately equal to √8, indicating the average spread of the observations from the mean. These functions are provided by the NumPy library, which is tightly integrated with SageMath and is widely used for statistical analysis and scientific computing.

Linear algebra

Linear algebra is the branch of mathematics that studies vectors, matrices, and systems of linear equations. It forms the mathematical foundation of modern scientific computing and underpins numerous fields, including computer graphics, optimization, signal processing, robotics, and scientific simulation. In recent years, its importance has grown even further, as much of modern artificial intelligence and quantum computing can be viewed as linear algebra in disguise—from neural network operations and tensor computations to quantum states and quantum gates, the underlying mathematics is fundamentally based